

Lab4实验报告

小组成员：2311858 许哲涵，2311576 赵宁，2313650 赵悦蛟

一、实验目的

- 了解虚拟内存管理的基本结构，掌握虚拟内存的组织与管理方式
- 了解内核线程创建/执行的管理过程
- 了解内核线程的切换和基本调度过程

二、实验内容

(一)本实验中重要的知识点

1.**虚拟内存**:虚拟内存是程序或CPU视角所看到的内存地址空间，它与物理内存并非直接对应，而是通过操作系统建立的内存映射关系进行转换。其核心作用体现在三个层面：首先是内存地址虚拟化，通过页表机制限制软件的访问空间，实现内存保护，防止越界访问；其次是实现按需分页，仅在程序实际访问某块虚拟内存时才分配物理内存并建立映射，提高内存利用率；最后是为页换入换出技术提供基础，将不常访问的数据暂存硬盘以腾出内存空间，从而给用户程序提供远大于实际物理内存的虚拟空间，支持更多程序并发运行。

2.**多级页表与虚拟存储实现**:在sv39这类采用多级页表的系统中，通过分解页表索引（如PDX1, PDX0, PTX）来管理庞大的地址空间。实现虚拟存储的关键在于动态管理页表映射，主要接口包括 `page_insert` 建立映射和 `page_remove` 删除映射。这要求将页表本身常驻内存并可修改。通过精细控制页表项，可以为不同内存段（如.text、.rodata、.data）设置不同的访问权限（读、写、执行），从而增强系统安全性。在初始化后对内核空间进行重映射时，可能需要创建全新的页表并切换satp寄存器来完成。

3.**进程与线程**:进程是正在运行的程序实例，它不仅包含静态代码，还包括运行时的堆栈、寄存器等动态状态，是操作系统进行资源分配的基本单位。线程则是进程内更轻量的执行流，共享同一进程的绝大部分资源（如内存空间），但拥有独立的CPU执行状态，是操作系统调度的基本单位。引入进程使得程序能够动态启动和管理，实现了资源的隔离与复用。在多核CPU背景下，多进程/线程能充分利用计算资源。即使在单核时代，通过分时技术也能实现多个进程的并发执行，提升系统整体利用率。

4.**进程上下文与切换机制**:进程上下文是进程在某一时刻的CPU运行现场，主要包括寄存器状态。进程切换时，需要保存当前进程的上下文，并恢复下一个进程的上下文，以确保进程能从中断点正确继续执行。在实际实现中，并非保存所有寄存器，而是巧妙地利用编译器对函数调用的约定，仅保存被调用者保存寄存器（如ra、sp、s0-s11），调用者保存寄存器由编译器在函数调用前后自动处理，从而减少开销。上下文信息保存在进程控制块（`proc_struct`）的context结构中。

(二)OS原理中很重要，但在实验中没有对应上的知识点

1.**虚拟内存中的页面置换算法**:在本实验中，都只涉及了页表的建立和映射（如 `page_insert`），并未真正实现当物理内存不足时，如何选择被换出的页面以及如何执行换入换出操作。

2.**进程间通信机制**:在真实的操作系统中，进程或线程之间通常需要协作，这就涉及到进程间通信。原理中重要的IPC机制，如信号量、管道、消息队列、共享内存等，在本次实验中均未涉及。实验中的内核线程是独立运行的，没有展现出它们之间如何进行数据交换或同步。

练习0：填写已有实验

练习1：分配并初始化一个进程控制块（需要编码）

`alloc_proc`函数 编写：

主要初始化内容包括：

```
proc->state = PROC_UNINIT; // 进程状态设为未初始化
proc->pid = -1; // 进程ID设为-1（未分配状态）
proc->runs = 0; // 运行次数初始化为0
proc->kstack = 0; // 内核栈地址设为0
proc->need_resched = 0; // 不需要重新调度
proc->parent = NULL; // 父进程指针设为NULL
proc->mm = NULL; // 内存管理结构设为NULL
memset(&(proc->context), 0, sizeof(struct context)); // 上下文结构清零
proc->tfs = 0; // 陷阱帧指针设为NULL
proc->pgdir = boot_pgdir; // 使用内核页目录表
proc->flags = 0; // 进程标志设为0
memset(proc->name, 0, PROC_NAME_LEN + 1); // 进程名清零
```

设计实现过程

1. **分配内存**：使用 `kmalloc` 分配 `struct proc_struct` 大小的内存空间
2. **检查分配结果**：确保内存分配成功后再进行初始化
3. **逐字段初始化**：
 - 将状态设为 `PROC_UNINIT` 表示进程尚未初始化完成
 - 将 `pid` 设为 -1 表示尚未分配有效进程ID
 - 数值型字段初始化为0，指针型字段初始化为NULL
 - 使用 `memset` 对结构体字段进行清零操作
 - 页目录设置为内核页目录 `boot_pgdir`
4. **返回结果**：返回初始化后的进程控制块指针

这样的初始化确保了新分配的进程控制块处于一个确定的安全状态，避免了未初始化内存可能带来的问题。

`struct context context` 的含义和作用

含义：`struct context` 是进程的上下文结构，用于保存进程切换时需要保护的寄存器状态。通常包括：

- 通用寄存器（`eax`, `ebx`, `ecx`, `edx`, etc.）
- 栈指针（`esp`）
- 基址指针（`ebp`）
- 程序计数器（`eip`）等

作用：在进程切换时，操作系统需要保存当前进程的执行上下文，并恢复下一个进程的上下文。`context` 结构就是用于保存和恢复这些寄存器状态的。当进行进程调度时：

1. 将当前进程的寄存器状态保存到其 `context` 结构中

2. 从下一个进程的 `context` 结构中恢复寄存器状态
3. 实现进程的透明切换

`struct trapframe *tf` 的含义和作用

含义： `struct trapframe` 是陷阱帧结构，用于保存中断或异常发生时的完整处理器状态。它包含了：

- 所有通用寄存器
- 段寄存器
- 错误代码
- 中断号
- 栈指针等完整状态

作用：

1. **中断处理：** 当发生中断或异常时，CPU自动将当前状态压栈形成陷阱帧
2. **进程执行：** 对于新创建的进程，`tf` 指向的陷阱帧决定了进程第一次执行时的初始状态
3. **上下文保存：** 在系统调用或中断处理过程中，`tf` 保存了进入内核前的用户态状态
4. **状态恢复：** 从内核返回用户态时，根据 `tf` 恢复进程的执行环境

练习2：为新创建的内核线程分配资源（需要编码）

在 `kern/process/proc.c` 中实现 `do_fork` 函数，为新创建的内核线程分配资源。以下是完整的设计实现：

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;

    // 1. 调用alloc_proc函数以分配一个proc_struct。
    proc = alloc_proc();
    if (proc == NULL) {
        goto fork_out;
    }

    // 2. 调用setup_kstack函数为子进程分配内核栈。
    if (setup_kstack(proc) != 0) {
        goto bad_fork_cleanup_proc;
    }

    // 3. 根据clone_flag，调用copy_mm以复制或共享内存管理结构(mm)。
    if (copy_mm(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_kstack;
    }
}
```

```

// 4. 调用copy_thread函数以设置proc_struct中的tf（陷阱帧）和context（上下文）。
copy_thread(proc, stack, tf);

// 5. 将proc_struct插入哈希链表和进程链表。
bool intr_flag;
local_intr_save(intr_flag); // 禁用中断
{
    proc->pid = get_pid();
    hash_proc(proc);
    list_add(&proc_list, &(proc->list_link));
    nr_process++;
}
local_intr_restore(intr_flag); // 启用中断

// 6. 调用wakeup_proc函数使新的子进程进入可运行状态。
wakeup_proc(proc);

// 7. 使用子进程的PID设置返回值。
ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

设计实现过程说明

执行步骤分析：

- 分配进程控制块：**调用 `alloc_proc()` 分配并初始化基本的进程控制块结构。
- 设置父进程：**将新进程的父进程指针指向当前正在执行的进程。
- 分配内核栈：**通过 `setup_kstack()` 为进程分配内核栈空间，这是进程执行时需要的栈内存。
- 复制内存管理信息：**对于内核线程，`copy_mm()` 通常返回0，因为内核线程共享内核地址空间。
- 复制执行上下文：**`copy_thread()` 函数负责：
 - 设置新进程的陷阱帧（trapframe），使其能够正确开始执行
 - 初始化上下文（context），特别是设置eip指向内核线程入口点
- 分配进程ID并加入进程列表：**在关中断保护下：
 - 调用 `get_pid()` 分配唯一PID
 - 将进程加入哈希表和进程链表
 - 增加进程计数
- 唤醒进程：**调用 `wakeup_proc()` 将进程状态设为就绪，等待调度执行。
- 错误处理：**在各个步骤失败时，正确地释放已分配的资源。

问题回答：ucore是否做到给每个新fork的线程一个唯一的id?

是的，ucore确实做到了给每个新fork的线程一个唯一的id。

分析和理由：

1. PID分配机制：

- 在 `get_pid()` 函数中，ucore维护了两个静态变量：`last_pid` 和 `next_safe`
- 系统通过遍历现有的进程列表来确保分配的PID不会重复

2. 保护机制：

- PID分配过程在关中断环境下进行，防止并发访问导致重复分配
- 使用 `local_intr_save()` 和 `local_intr_restore()` 来保护临界区

3. 唯一性保证：

```
- // 在关中断保护下分配PID
local_intr_save(intr_flag);
{
    proc->pid = get_pid(); // 这个函数保证返回唯一PID
    // ... 其他操作
}`` local_intr_restore(intr_flag);`
```

- ucore限制了PID的有效范围，避免无限制增长
- 当PID达到上限时会回绕并重新查找可用PID

4. 实际验证：

- 通过实验调试可以观察到，每次调用 `do_fork` 都会返回不同的PID
- 进程列表中的每个进程都有唯一的PID标识

因此，ucore通过合理的PID管理算法和并发保护机制，确保了每个新创建的内核线程都能获得全局唯一的进程ID。

练习3：编写proc_run 函数（需要编码）

`proc_run` 函数负责进程切换，其核心代码如下：

```
void proc_run(struct proc_struct *proc) {
    if (proc != current) {
        // 1. 禁用中断
        bool intr_flag;
        struct proc_struct *prev = current, *next = proc;
        local_intr_save(intr_flag);
        {
            // 2. 切换当前进程
            current = proc;
            // 3. 切换页表
            lcr3(next->cr3);
            // 4. 上下文切换
            switch_to(&(prev->context), &(next->context));
        }
    }
}
```

```
// 5. 允许中断
local_intr_restore(intr_flag);
}
}
```

问题回答：创建且运行了几个内核线程？

在本实验的执行过程中，创建且运行了 2 个内核线程。

详细分析：

1. idleproc（第0个内核线程）

- **创建时机：**在 `proc_init()` 函数中首先创建
- **作用：**系统空闲时运行的线程，当没有其他就绪线程时调度执行
- **特点：**
 - PID = 0
 - 名称为 "idle"
 - 不断循环，调用 `schedule()` 进行进程调度
 - 是所有其他线程的祖先

2. initproc（第1个内核线程）

- **创建时机：**在 `proc_init()` 中通过 `kernel_thread(init_main, NULL, 0)` 创建
- **作用：**执行初始化工作，打印 "Hello World!!" 信息
- **特点：**
 - PID = 1
 - 名称为 "init"
 - 执行 `init_main` 函数
 - 是第一个实际执行具体任务的内核线程

执行流程验证：

从实验代码可以看到：

```
// proc_init 函数中的关键代码
void proc_init(void) {
    // 创建第0个内核线程 idle
    idleproc = alloc_proc();
    idleproc->pid = 0;
    idleproc->state = PROC_RUNNABLE;
    idleproc->kstack = (uintptr_t)bootstack;
    idleproc->need_resched = 1;
    set_proc_name(idleproc, "idle");
    nr_process++;
    // 创建第1个内核线程 init
    int pid = kernel_thread(init_main, NULL, 0);
}
```

```
if (pid <= 0) {
    panic("create init_main failed.\n");
}
initproc = find_proc(pid);
set_proc_name(initproc, "init");

// 验证
assert(idleproc != NULL && idleproc->pid == 0);
assert(initproc != NULL && initproc->pid == 1);
}
```

运行过程：

1. 系统启动后首先运行 `idleproc`
2. 调度器发现 `initproc` 就绪后，通过 `proc_run` 切换到 `initproc`
3. `initproc` 执行 `init_main` 函数，打印 "Hello World!!"
4. 执行完成后，`initproc` 调用 `do_exit` 退出
5. 系统回到 `idleproc` 继续运行

运行效果：

`make qemu` 命令：

```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/lab4_final
end    0xc020d4f0 (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaoffset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:412:
  process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```

make grade 命令:


```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/lab4_final
riscv64-unknown-elf-ld: removing unused section '.text.sprintf' in file 'obj/lib
bs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.strncpy' in file 'obj/lib
s/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strfind' in file 'obj/lib
s/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strtol' in file 'obj/lib
s/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.memmove' in file 'obj/lib
s/string.o'
gmake[1]: 进入目录“/home/ubuntu/lab4_final” + cc kern/init/entry.S + cc kern/ini
t/init.c + cc kern/libs/readline.c + cc kern/libs/stdio.c + cc kern/debug/kdebug
.c + cc kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/clock.c +
cc kern/driver/console.c + cc kern/driver/dtb.c + cc kern/driver/intr.c + cc ke
rn/driver/picirq.c + cc kern/trap/trap.c + cc kern/trap/trapentry.S + cc kern/mm
/default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc
kern/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc k
ern/schedule/sched.c + cc libs/hash.c + cc libs/printfmt.c + cc libs/string.c +
ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/u
core.img gmake[1]: 离开目录“/home/ubuntu/lab4_final”
-check alloc proc: OK
-check initproc: OK
Total Score: 30/30
```

扩展练习 Challenge:

1.说明语句

`local_intr_save(intr_flag); ... local_intr_restore(intr_flag);` 是如何实现开关中断的?

在操作系统内核中, `local_intr_save(intr_flag)` 和 `local_intr_restore(intr_flag)` 是用于临时开关中断的配对操作, 其核心目的是在执行临界区代码 (如进程切换) 时禁止外部中断干扰, 保证操作的原子性。以下是其实现原理的详细说明:

(1) `local_intr_save(intr_flag)`: 禁用中断并保存当前中断状态

该函数的作用是**关闭当前CPU的中断响应**, 同时将关闭前的中断状态 (是否允许中断) 保存到 `intr_flag` 变量中, 以便后续恢复。

实现逻辑 (以RISC-V架构为例) :

- 通过读取CPU状态寄存器 (如 `sstatus`) 获取当前中断使能标志 (例如 `SIE` 位, Supervisor Interrupt Enable)。
- 将该标志位的值保存到 `intr_flag` 中 (记录中断原来是否处于开启状态)。
- 清除状态寄存器中的中断使能标志 (如清除 `SIE` 位), 使CPU不再响应外部中断 (硬件中断、时钟中断等)。

伪代码示意：

```
void local_intr_save(unsigned long *intr_flag) {  
    // 读取当前CPU状态寄存器（如sstatus）  
    unsigned long sstatus = csrr(sstatus);  
    // 保存中断使能状态（提取SIE位）  
    *intr_flag = sstatus & SSTATUS_SIE;  
    // 关闭中断（清除SIE位）  
    csrwr(sstatus, sstatus & ~SSTATUS_SIE);  
}
```

(2) local_intr_restore(intr_flag)：恢复中断状态

该函数的作用是**根据之前保存的中断状态**（`intr_flag`），恢复CPU的中断响应能力。如果中断在关闭前是开启的，则重新开启；如果原本就是关闭的，则保持关闭。

实现逻辑（以RISC-V架构为例）：

- 读取当前CPU状态寄存器（如 `sstatus`）。
- 根据 `intr_flag` 中保存的原始状态，决定是否重新设置中断使能标志（`SIE` 位）：
 - 若 `intr_flag` 非0（表示中断原来处于开启状态），则在状态寄存器中置位 `SIE`，重新允许中断。
 - 若 `intr_flag` 为0（表示中断原来已关闭），则保持 `SIE` 位为0，不改变状态。

伪代码示意：

```
void local_intr_restore(unsigned long intr_flag) {  
    // 读取当前CPU状态寄存器  
    unsigned long sstatus = csrr(sstatus);  
    if (intr_flag) {  
        // 恢复中断使能（置位SIE位）  
        csrwr(sstatus, sstatus | SSTATUS_SIE);  
    } else {  
        // 保持中断关闭（SIE位仍为0）  
        csrwr(sstatus, sstatus & ~SSTATUS_SIE);  
    }  
}
```

(3) 为何在进程切换中需要开关中断？

进程切换（`switch_to`）是内核的核心临界操作，需要保证原子性：

- 若切换过程中被中断（如时钟中断），可能导致进程上下文混乱（如寄存器状态未保存完整）。
- `local_intr_save` 确保切换过程中CPU不响应外部中断，避免干扰；`local_intr_restore` 则在切换完成后恢复中断响应，保证系统能继续处理外部事件（如IO中断、时钟调度）。

总结

- `local_intr_save` 通过读取并保存CPU中断状态，再关闭中断，实现“禁用中断”。
- `local_intr_restore` 根据保存的状态，恢复中断使能标志，实现“恢复中断”。
- 两者配合确保临界区代码（如进程切换）的原子性，是内核同步的基础操作。
不同架构（如x86、ARM）的寄存器和标志位名称可能不同（例如x86用 `eflags` 寄存器的 `IF` 位），但核心逻辑一致：保存状态→关闭中断→执行临界区→恢复状态。

2.深入理解不同分页模式的工作原理（思考题）

`get_pte()` 函数（位于 `kern/mm/pmm.c`）用于在页表中查找或创建页表项，从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下，是实现虚拟内存与物理内存之间映射关系非常重要的内容。

(1)`get_pte()`函数中有两段形式类似的代码，结合sv32，sv39，sv48的异同，解释这两段代码为什么如此相像。

两段相似代码的原因：页表结构的“多级递推”共性

RISC-V 的 sv32、sv39、sv48 均采用**多级页表**机制，不同模式的差异主要在于页表级数和虚拟地址的划分方式，但核心逻辑是一致的：**通过多级页表项逐级索引，最终定位到物理页**。具体来说：

分页模式	页表级数	虚拟地址划分（从高到低）	每级页表索引位宽
sv32	2 级	一级页表索引（10 位） + 二级页表索引（10 位） + 页内偏移（12 位）	10 位
sv39	3 级	一级（9 位） + 二级（9 位） + 三级（9 位） + 页内偏移（12 位）	9 位
sv48	4 级	一级（9 位） + 二级（9 位） + 三级（9 位） + 四级（9 位） + 页内偏移（12 位）	9 位

`get_pte()` 函数的代码片段对应**多级页表的逐级解析过程**。以当前代码为例（推测为 sv32 模式，2 级页表）：

1. 第一段代码处理**一级页表项 (pdep1)**：从页目录 (pgdir) 中取出一级索引 (PDX1(la)) 对应的页表项，若未有效 (!PTE_V)，则分配新页作为二级页表，并初始化页表项。
2. 第二段代码处理**二级页表项 (pdep0)**：以一级页表项指向的物理地址作为二级页表基址，取出二级索引 (PDX0(la)) 对应的页表项，若未有效，则分配新页作为最终的页表（或物理页），并初始化页表项。

两段代码相似的核心原因是：**多级页表的每一级解析逻辑完全一致**——都是“检查当前级页表项是否有效→若无效且需要创建，则分配新页表→初始化页表项并标记有效→进入下一级页表”。无论是 sv32 的 2 级、sv39 的 3 级还是 sv48 的 4 级，每一级的处理步骤完全相同，因此代码结构必然相似。如果扩展到 sv39 或 sv48，只需在相同逻辑上增加对应级数的循环或代码段即可（本质是重复相同的“级解析”逻辑）。

(2)目前`get_pte()`函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

`get_pte()` 将“查找页表项”和“分配缺失的页表”合并在一个函数中，这种设计的优劣需结合使用场景判断：

1.优势：简化临界路径代码

在操作系统中，`get_pte()` 的核心使用场景是**创建虚拟地址映射**（如 `mmap`、页错误处理）。此时的逻辑必然是：“查找页表项→若不存在则分配页表→返回页表项”。将这两个步骤合并为一个函数，可以：

- 减少函数调用开销（无需在外部判断后再调用分配函数）；
- 避免重复代码（查找和分配的逻辑紧密关联，拆分后需重复传递参数和处理页表索引）；
- 保证操作的原子性（例如在分配页表时需要加锁，合并后锁的范围更清晰）。

2.劣势：灵活性降低

如果存在**仅需查找页表项而不分配**的场景（如查询地址映射是否存在），合并后的函数需要通过 `create` 参数控制行为（`create=false` 时只查找不分配），虽然可行，但语义上不够直观。

3. 是否需要拆分？

不建议拆分，理由如下：

- 操作系统中“查找页表”的主要目的是为了“创建映射”，仅查找不分配的场景较少（可通过 `create=false` 支持）；
- 多级页表的分配逻辑与查找逻辑高度耦合（每一级的查找都可能需要分配下一级页表），拆分后会导致代码冗余（例如每一级都需要判断是否分配）；
- 从接口设计看，`get_pte(pgdir, la, create)` 通过参数明确控制行为，兼顾了两种需求，且符合“单一职责”（目标是获取页表项，分配是实现该目标的手段）。

总结

1. 两段代码相似是因为**多级页表的每一级解析逻辑完全一致**（检查有效性→分配页表→初始化），与 `sv32/sv39/sv48` 的级数差异无关，仅需按级数重复该逻辑。
2. 功能合并的设计更适合操作系统的实际场景，兼顾了效率和简洁性，无需拆分。